부록 · 파이썬으로 DB 다루기

pymongo — 찾고 · 세고 · 묶고 · 고치기

2-3 에서 load_logs.py 로 넣은 접속 로그 1,000건(logdb.access_logs)을 파이썬에서 다룹니다. 먼저 2-3 의 de-mongo 를 켜고 load_logs.py 를 한 번 돌려 두세요.

준비

```bash
docker start de-mongo
docker exec de-mongo mongosh --quiet --eval "db.runCommand({ ping: 1 }).ok"   # 1 이면 준비 완료
cd ~/data-eng/week2 && source .venv/bin/activate
python3 load_logs.py          # 2-3 에서 만든 파일 — 몇 번 돌려도 같은 1,000건
```

코드

```python
# py_mongo.py — pymongo 로 찾고·세고·묶고·고치기 (2-3 의 de-mongo · logdb.access_logs)
from pymongo import MongoClient, ASCENDING, DESCENDING
import pandas as pd

client = MongoClient('mongodb://localhost:27017', serverSelectionTimeoutMS=5000)
col = client['logdb']['access_logs']

# 찾기 — 조건 · 가져올 칸(1) · 정렬 · 개수
for doc in (col.find({'level': 'ERROR'}, {'_id': 0, 'ts': 1, 'path': 1, 'error.code': 1})
               .sort('ts', ASCENDING).limit(3)):
    print(doc)

# 세기
print('ERROR', col.count_documents({'level': 'ERROR'}), '건')

# 묶기(aggregate) — 경로별 ERROR 수, 많은 순
pipeline = [
    {'$match': {'level': 'ERROR'}},
    {'$group': {'_id': '$path', 'n': {'$sum': 1}}},
    {'$sort': {'n': -1, '_id': 1}},
]
for row in col.aggregate(pipeline):
    print(row)

# 고치기 — 결제 경로 오류에 확인 표시 달기
r = col.update_many({'level': 'ERROR', 'path': '/api/pay'}, {'$set': {'checked': True}})
print('맞은', r.matched_count, '/ 바뀐', r.modified_count)

# 색인 — 자주 찾는 칸에 만든다. 이미 있으면 그대로 두고 이름만 돌려준다
print('색인:', col.create_index([('level', ASCENDING), ('ts', DESCENDING)]))

# pandas 로 — 커서를 리스트로 받아 DataFrame
df = pd.DataFrame(list(col.find({'source': 'load_logs'}, {'_id': 0, 'level': 1, 'status':
1})))
print(df.groupby('level').size())
```

```bash
python3 py_mongo.py
```

실행 결과 (실측)

```text
{'ts': datetime.datetime(2026, 8, 31, 9, 0, 9), 'path': '/', 'error': {'code': 'E64'}}
{'ts': datetime.datetime(2026, 8, 31, 9, 0, 18), 'path': '/', 'error': {'code': 'E35'}}
{'ts': datetime.datetime(2026, 8, 31, 9, 0, 21), 'path': '/api/pay', 'error': {'code': 'E67'}}
ERROR 201 건
{'_id': '/api/pay', 'n': 46}
{'_id': '/mypage', 'n': 42}
{'_id': '/search', 'n': 42}
{'_id': '/login', 'n': 40}
{'_id': '/', 'n': 31}
맞은 46 / 바뀐 46
색인: level_1_ts_-1
level
ERROR    201
INFO     586
WARN     213
dtype: int64
```

두 번째로 돌리면 「맞은 46 / 바뀐 0」이 나옵니다. 이미 checked 가 True 라 바꿀 것이 없었다는 뜻이고, 정상입니다(실측). 나머지 줄은 같습니다.

짚을 점

- find(조건, 가져올 칸) — 가져올 칸에 1 을 적은 것만 옵니다. _id 는 0 으로 적어야 빠집니다. 'error.code' 처럼 점으로 안쪽 칸을 고릅니다.
- count_documents — 조건에 맞는 문서 수입니다. 조건 없이 전부 세려면 {} 를 넘깁니다.
- aggregate — $match(거르기) → $group(묶기) → $sort(정렬)를 차례로 통과시킵니다. SQL 의 WHERE → GROUP BY → ORDER BY 와 같습니다. 같은 수가 있으면 순서가 흔들리지 않게 '_id' 로 한 번 더 정렬했습니다.
- update_many 의 결과 — matched_count 는 조건에 맞은 수, modified_count 는 실제로 바뀐 수입니다. 둘이 다르면 이미 그 값이었던 문서가 있다는 뜻입니다.
- serverSelectionTimeoutMS=5000 — 컨테이너가 꺼져 있을 때 기본 30초 대신 5초 만에 실패합니다. 부록 「실습 트러블슈팅」의 시간 초과 항목과 같은 상황입니다.
